**Task 1: Data Preprocessing and Exploratory Analysis (Titanic Dataset)**

- **Exam Task:**
    1. Load the Titanic dataset using Pandas and handle missing values by imputing missing `Age` values with the median
    2. Filter the dataset to extract all surviving passengers (`Survived == 1`) and compute the average and maximum `Fare` using NumPy operations
    3. Using Seaborn or Matplotlib, create two visualizations: a **survival count bar plot** and an **Age vs. Fare scatter plot**

```
# Task 1: Data Preprocessing and Exploratory Analysis

# 1. Import Libraries

import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


# 2. Load the Titanic Dataset

# Upload your Titanic.csv file to Google Colab first

from google.colab import files

uploaded = files.upload()


df = pd.read_csv("Titanic.csv")


# Display first five rows

print("First Five Rows:")

print(df.head())
```

```
# 3. Handle Missing Age Values Using Median
print("\nMissing Age Values Before Imputation:")
print(df["Age"].isnull().sum())

df["Age"] = df["Age"].fillna(df["Age"].median())

print("\nMissing Age Values After Imputation:")
print(df["Age"].isnull().sum())

# 4. Extract Surviving Passengers
survivors = df[df["Survived"] == 1]

print("\nSurviving Passengers:")
print(survivors.head())

# Calculate Average and Maximum Fare Using NumPy
average_fare = np.mean(survivors["Fare"].dropna().to_numpy())
maximum_fare = np.max(survivors["Fare"].dropna().to_numpy())

print("\nAverage Fare of Survivors:", average_fare)
print("Maximum Fare of Survivors:", maximum_fare)

# 5. Visualization 1: Survival Count Bar Plot
plt.figure(figsize=(7, 5))
```

```
sns.countplot(data=df, x="Survived")


plt.title("Titanic Passenger Survival Count")

plt.xlabel("Survived (0 = No, 1 = Yes)")

plt.ylabel("Number of Passengers")

plt.show()


# 6. Visualization 2: Age vs Fare Scatter Plot

plt.figure(figsize=(8, 5))


sns.scatterplot(data=df, x="Age", y="Fare", hue="Survived")


plt.title("Age vs Fare of Titanic Passengers")

plt.xlabel("Age")

plt.ylabel("Fare")

plt.legend(title="Survived", labels=["No", "Yes"])

plt.show()
```

---

**Task 2: Simple Linear Regression using OLS (Auto MPG Dataset)**

- **Exam Task:**
    1. Load the Auto MPG dataset, extract `Horsepower` as the predictor and `MPG` as the target variable, and drop/impute any null rows.

2. Add the intercept/bias term and calculate the Ordinary Least Squares (OLS) closed-form weights analytically using the normal equation ($\hat{w} = (X^T X)^{-1} X^T y$).
3. Predict the target values and compute the **Mean Squared Error (MSE)** and $R^2$ **score**
4. Plot the fitted regression line over the data scatter plot

# Task 2: Simple Linear Regression using OLS

```
# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error, r2_score

# 2. Load the Auto MPG Dataset
# Upload your CSV file in Google Colab
from google.colab import files
uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0])

print("First Five Rows:")
print(df.head())

# If the dataset uses the original UCI column names,
```

```
# rename them to Horsepower and MPG
df.columns = df.columns.str.strip()

# Handle datasets with standard UCI Auto MPG column names
if "horsepower" in df.columns:
    df.rename(columns={"horsepower": "Horsepower"}, inplace=True)
if "mpg" in df.columns:
    df.rename(columns={"mpg": "MPG"}, inplace=True)

# 3. Select Predictor and Target
df["Horsepower"] = pd.to_numeric(df["Horsepower"], errors="coerce")
df["MPG"] = pd.to_numeric(df["MPG"], errors="coerce")

# Drop rows containing missing values in required columns
df = df.dropna(subset=["Horsepower", "MPG"])

X = df["Horsepower"].to_numpy().reshape(-1, 1)
y = df["MPG"].to_numpy()

# 4. Add Intercept/Bias Term
X_bias = np.c_[np.ones(X.shape[0]), X]

print("\nShape of X with bias:", X_bias.shape)
```

```
# 5. Calculate OLS Weights using Normal Equation
# w = (X^T X)^(-1) X^T y
weights = np.linalg.pinv(X_bias.T @ X_bias) @ X_bias.T @ y

intercept = weights[0]
slope = weights[1]

print("\nOLS Weights:")
print("Intercept:", intercept)
print("Slope:", slope)

# 6. Predict Target Values
y_pred = X_bias @ weights

print("\nFirst 10 Predicted MPG Values:")
print(y_pred[:10])

# 7. Calculate MSE and R2 Score
mse = mean_squared_error(y, y_pred)
r2 = r2_score(y, y_pred)

print("\nMean Squared Error (MSE):", mse)
print("R2 Score:", r2)
```

```
# 8. Plot Regression Line over Scatter Plot
plt.figure(figsize=(8, 5))

plt.scatter(X, y, alpha=0.6, label="Actual Data")

# Sort values to draw a smooth regression line
sorted_indices = np.argsort(X[:, 0])
X_sorted = X[sorted_indices]
y_pred_sorted = y_pred[sorted_indices]

plt.plot(X_sorted, y_pred_sorted, color="red",
        linewidth=2, label="OLS Regression Line")

plt.xlabel("Horsepower")
plt.ylabel("Miles Per Gallon (MPG)")
plt.title("Simple Linear Regression: Horsepower vs MPG")
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

---

**Task 3: Multiple Linear Regression from Scratch (CarDekho Dataset)**

- **Exam Task:**

1. Load the CarDekho dataset, select at least two numerical predictors (e.g., `Present_Price`, `Kms_Driven`) and encode one categorical variable (e.g., `Fuel_Type`)
2. Standardize numerical features ($z = \frac{x-\mu}{\sigma}$) and split into an 80:20 train-test split
3. Prepend a bias column of $1.0$ and compute parameter weights $\hat{\beta} = (X^T X)^{-1} X^T y$ using purely **NumPy matrix operations** (no scikit-learn regression functions)
4. Predict test set car selling prices and compute the **Residual Sum of Squares (RSS)** and $R^2$.

```
# ========================================

# Task 3: Multiple Linear Regression from Scratch

# CarDekho Dataset

# ========================================


# 1. Import Libraries

import pandas as pd

import numpy as np


# 2. Load Dataset

from google.colab import files

uploaded = files.upload()


df = pd.read_csv(list(uploaded.keys())[0])


print("First Five Rows:")
```

```
print(df.head())


# 3. Select Features and Target
# Numerical predictors: Present_Price, Kms_Driven
# Categorical predictor: Fuel_Type
# Target: Selling_Price

df = df[["Present_Price", "Kms_Driven", "Fuel_Type", "Selling_Price"]]


# Convert numerical columns to numeric
df["Present_Price"] = pd.to_numeric(
    df["Present_Price"], errors="coerce"
)
df["Kms_Driven"] = pd.to_numeric(
    df["Kms_Driven"], errors="coerce"
)
df["Selling_Price"] = pd.to_numeric(
    df["Selling_Price"], errors="coerce"
)

# Handle missing values
df = df.dropna()

# 4. Encode Categorical Variable
```

```
df = pd.get_dummies(
    df, columns=["Fuel_Type"], drop_first=True, dtype=int
)

print("\nDataset After Encoding:")
print(df.head())

# Separate predictors and target
X = df.drop("Selling_Price", axis=1).to_numpy(dtype=float)
y = df["Selling_Price"].to_numpy(dtype=float)

# 5. Standardize Numerical Features
# z = (x - mean) / standard deviation
# Standardize the first two columns only

X_num = X[:, :2]

mean = X_num.mean(axis=0)
std = X_num.std(axis=0)

# Avoid division by zero
std[std == 0] = 1

X[:, :2] = (X_num - mean) / std
```

```
# 6. Split Dataset into 80:20 Train-Test Split

np.random.seed(42)


indices = np.random.permutation(len(X))

split_index = int(0.8 * len(X))


train_idx = indices[:split_index]

test_idx = indices[split_index:]


X_train = X[train_idx]

X_test = X[test_idx]


y_train = y[train_idx]

y_test = y[test_idx]


print("\nTraining Samples:", len(X_train))

print("Testing Samples:", len(X_test))


# 7. Add Bias Column

X_train_bias = np.c_[np.ones(X_train.shape[0]), X_train]

X_test_bias = np.c_[np.ones(X_test.shape[0]), X_test]


# 8. Calculate Weights using NumPy Normal Equation
```

```
# w = (X^T X)^(-1) X^T y
# Pseudoinverse handles singular or near-singular matrices

weights = np.linalg.pinv(
    X_train_bias.T @ X_train_bias
) @ X_train_bias.T @ y_train

print("\nCalculated Weights:")
print(weights)

# 9. Predict Test Set Selling Prices
y_pred = X_test_bias @ weights

print("\nActual Selling Prices:")
print(y_test[:10])

print("\nPredicted Selling Prices:")
print(y_pred[:10])

# 10. Calculate RSS and R2 Score
RSS = np.sum((y_test - y_pred) ** 2)

SS_total = np.sum((y_test - np.mean(y_test)) ** 2)
```

R2 = 1 - (RSS / SS_total) if SS_total != 0 else np.nan

print("\nResidual Sum of Squares (RSS):", RSS)

print("R2 Score:", R2)

---

**Task 4: Binary Logistic Regression & Fraud Evaluation (Credit Card Fraud Dataset)**

- **Exam Task:**
    1. Load a pre-sampled/balanced subset of credit card transaction data and standardize the numerical predictors1819.
    2. Implement the **Sigmoid function** $\sigma(z) = \frac{1}{1+e^{-z}}$ and train a Logistic Regression model.
    3. Generate class predictions using a decision threshold of $0.5$ on the test split.
    4. Compute and display the **Confusion Matrix**, **Precision**, **Recall**, and **F1-Score**, highlighting why Recall is critical in fraud detection

---

**Task 5: Bagging Ensemble Classifier from Scratch (Pima Indians Diabetes Dataset)**

- **Exam Task:**
    1. Load the Pima Indians Diabetes dataset and handle zero-value anomalies in features like `Glucose` or `BloodPressure`
    2. Write a custom bootstrap function using `np.random.choice` (sampling with replacement) to train an ensemble of $B = 10$ to $20$ unpruned `DecisionTreeClassifier` base learners
    3. Implement a custom aggregation function that applies **majority voting** to make final predictions on test samples.
    4. Compare the test set **Accuracy** and **Confusion Matrix** of your custom Bagging model against a standalone single Decision Tree.

```
# ========================================

# Task 5: Bagging Ensemble Classifier from Scratch
```

```
# Pima Indians Diabetes Dataset
# ========================================

# 1. Import Libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# 2. Load Dataset
from google.colab import files
uploaded = files.upload()

df = pd.read_csv(list(uploaded.keys())[0])

print("First Five Rows:")
print(df.head())

# 3. Handle Zero-Value Anomalies
# Zero is not a valid measurement for these features
zero_columns = [
    "Glucose",
    "BloodPressure",
```

```
    "SkinThickness",
    "Insulin",
    "BMI"
]

for col in zero_columns:
    df[col] = df[col].replace(0, np.nan)

# Impute missing/anomalous values with column median
for col in zero_columns:
    df[col] = df[col].fillna(df[col].median())

print("\nMissing Values After Imputation:")
print(df[zero_columns].isnull().sum())

# 4. Separate Features and Target
X = df.drop("Outcome", axis=1).to_numpy(dtype=float)
y = df["Outcome"].to_numpy(dtype=int)

# 5. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
```

```
    stratify=y
)

# 6. Custom Bootstrap Sampling Function
def bootstrap_sample(X, y):
    n_samples = X.shape[0]

    # Sampling with replacement
    indices = np.random.choice(
        n_samples,
        size=n_samples,
        replace=True
    )

    return X[indices], y[indices]

# 7. Train Bagging Ensemble
def train_bagging(X_train, y_train, B=10):
    models = []

    for i in range(B):
        X_boot, y_boot = bootstrap_sample(X_train, y_train)

        # Unpruned Decision Tree
```

```
        tree = DecisionTreeClassifier(
            random_state=100 + i
        )

        tree.fit(X_boot, y_boot)
        models.append(tree)

    return models

np.random.seed(42)

B = 10
models = train_bagging(X_train, y_train, B)

print("\nNumber of Trees in Ensemble:", len(models))

# 8. Custom Majority Voting Function
def majority_vote(models, X):
    # Each row contains predictions from all trees
    predictions = np.array([
        model.predict(X) for model in models
    ])

    final_predictions = []
```

```
    for i in range(predictions.shape[1]):
        values, counts = np.unique(
            predictions[:, i],
            return_counts=True
        )

        # In a tie, np.argmax selects the first class,
        # which is 0 for binary labels {0, 1}.
        final_predictions.append(values[np.argmax(counts)])

    return np.array(final_predictions)

# 9. Predict Using Custom Bagging
y_pred_bagging = majority_vote(models, X_test)

# 10. Train a Single Decision Tree
single_tree = DecisionTreeClassifier(random_state=42)
single_tree.fit(X_train, y_train)

y_pred_single = single_tree.predict(X_test)

# 11. Evaluate Custom Bagging
bagging_accuracy = accuracy_score(y_test, y_pred_bagging)
```

```
bagging_cm = confusion_matrix(y_test, y_pred_bagging)

print("\n===== Custom Bagging Results =====")
print("Accuracy:", bagging_accuracy)
print("Confusion Matrix:\n", bagging_cm)

# 12. Evaluate Single Decision Tree
single_accuracy = accuracy_score(y_test, y_pred_single)
single_cm = confusion_matrix(y_test, y_pred_single)

print("\n===== Single Decision Tree Results =====")
print("Accuracy:", single_accuracy)
print("Confusion Matrix:\n", single_cm)

# 13. Compare Accuracies
print("\n===== Accuracy Comparison =====")
print("Custom Bagging Accuracy:", bagging_accuracy)
print("Single Tree Accuracy:", single_accuracy)

if bagging_accuracy > single_accuracy:
    print("Bagging performs better on this test set.")
elif bagging_accuracy < single_accuracy:
    print("Single Decision Tree performs better on this test set.")
else:
```

print("Both models have equal accuracy on this test set.")

---

**Task 6: Support Vector Machine vs. Linear Classifier Boundary (Iris Dataset)**

- **Exam Task:**
    1. Load the Iris dataset, drop the `Id` column, and select two continuous features (e.g., `Petal Length` and `Petal Width`) for 2D visualization.
    2. Standardize features and split the data into train and test sets.
    3. Train a **Linear SVM** (`SVC(kernel='linear')`) and a **Logistic Regression** model on the same two features
    4. Identify and print the coordinates/indices of the **Support Vectors**
    5. Plot the decision boundaries of both classifiers on the same plot or side-by-side subplots to illustrate maximum margin separation vs. a standard linear boundary

---

**Task 7: PCA Dimensionality Reduction & Impact on Classification (Breast Cancer Dataset)**

- **Exam Task:**
    1. Load the Breast Cancer Wisconsin dataset, drop metadata columns (`id`, `Unnamed: 32`), and scale the 30 continuous features with `StandardScaler`
    2. Apply `PCA(n_components=2)` to compress the 30 features into $PC_1$ and $PC_2$, and print their individual and cumulative **explained variance ratios**.
    3. Create a 2D scatter plot of $PC_1$ vs. $PC_2$ color-coded by diagnosis (`Malignant` vs. `Benign`).
    4. Train a classifier (such as Logistic Regression) on the reduced 2-component dataset and report the test accuracy.

---

**Task 8: Graph-Based MST Clustering (Breast Cancer Wisconsin Dataset)**

- **Exam Task:**

1. Load the Breast Cancer dataset (or a sampled 100–150 record subset for rapid graph calculation), select numerical attributes, and standardize them
2. Construct a pairwise Euclidean distance matrix and compute the **Minimum Spanning Tree (MST)** using `scipy.sparse.csgraph.minimum_spanning_tree` (or `networkx`)
3. Partition the graph into $K = 2$ clusters by removing the single largest edge weight in the MST
4. Assign cluster labels to the two resulting connected components and evaluate clustering performance against ground truth labels using the **Adjusted Rand Index (ARI)** or **Silhouette Score**

---

**Task 9: CART Decision Tree vs. Linear Classification Boundary (Palmer Penguins Dataset)**

- **Exam Task:**
    1. Load the Palmer Penguins Extended dataset, drop rows with missing values, and select two features (`culmen_length_mm` and `flipper_length_mm`) with `species` as the target
    2. Train a `DecisionTreeClassifier` using the **Gini index** criterion and a baseline `LogisticRegression` model on the same 2D data
    3. Plot side-by-side subplots comparing the **axis-aligned rectangular decision boundary** of CART with the **linear decision boundary** of Logistic Regression
    4. Generate and display the **Classification Report** (Precision, Recall, F1-Score) for the CART model

---

**Task 10: DBSCAN Clustering & Outlier Detection (Mall Customer Segmentation Dataset)**

- **Exam Task:**
    1. Load the Mall Customer Segmentation dataset, extract numerical features (e.g., `Annual Income (k$)` and `Spending Score (1-100)`), and scale them appropriately using `StandardScaler`.
    2. Fit a `DBSCAN` model with chosen hyperparameter values (e.g., `eps` and `min_samples`).

3. Extract and display the number of identified customer clusters as well as the count of spatial noise/outlier points labeled as `-1`.
4. Create a scatter plot visualizing the clusters with distinct colors, explicitly highlighting the detected spatial outliers/noise points1.